■■ System Architecture & End-to-End Business Cycles

> ***D365FO Middleware Backend — Comprehensive Architecture & Process Flows***
>
> *Target Audience: Software Engineers, Integration Architects, Finance & Operations Teams.*

---

1. Executive Business Purpose & System Context

The **D365FO Middleware** serves as an intelligent integration and processing layer between **operational Excel workbooks / logistics software** and **Microsoft Dynamics 365 Finance & Operations (D365FO)**.

The Problem It Solves

Logistics operations involve hundreds of daily transactions across sea/air freight forwarding, inland fleet trucking, container yard services, driver custody advances, and vendor sub-contractor bills.

- **Before Middleware:** Manual re-keying into D365FO was slow, prone to accounting errors (wrong main accounts or cost centers), created month-end cut-off bottlenecks, and frequently failed due to OData API timeouts when posting large volumes.
- **With Middleware:** Operations staff upload Excel sheets. The middleware automatically validates financial dimensions, enforces month isolation rules, formats payloads, splits large files into managed 1000-line batches, and posts asynchronously via resilient queues into D365FO.

---

2. Architecture Overview

```
[ Operational Excel Workbooks / APIs ]
■
▼
[ NestJS REST API Gateway (v1) ]
■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Auth, User & RBAC Safeguards ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
[ Entry Processor Engine & Factory ]
(Enriches, Validates Dimensions & Applies Month Rules)
■
▼
[ MongoDB Persistence (CQRS Store) ]
Status: PENDING BATCH
■
▼
[ BullMQ Asynchronous Queues ]
(Redis-Backed Background Worker Distribution)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ ■ ■ ■
[ FTI Queue ] [ Vendor Queue ] [ GL Queue ] [ Cust-Pay Queue ]
■ ■ ■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■
```

```
[ D365FO OData Client ]
(Opossum Circuit Breaker + Axios Exponential Retry)
■
▼
[ Microsoft Dynamics 365 Finance & Operations ]
(FreeTextInvoiceTable, LedgerJournalTable, CustPayTable)
```

---

3. End-to-End Business Cycles

The middleware operates on **4 Core Business Cycles**:

■ Cycle 1: Ingestion, Dimension Validation & Batching Cycle

Real-World Business Scenario

An operations coordinator finishes processing the monthly freight invoices or trucking manifest in Excel and uploads the file via API.

```
[User Uploads Excel]
■
▼
[Parse Excel Workbook] ■■■> Extract Raw Rows (EntryNameRawData)
■
▼
[EntryProcessorFactory] ■■> Select Specific Domain Processor
■
▼
[Dimension Validation] ■■■> Parse ACCOUNTDISPLAYVALUE (MainAccount|BU|CC|Activity...)
■ Verify Main Account active & dimensions valid against D365 Cache
■
▼
[Month Isolation Rule] ■■■> Ensure single batch contains lines for ONE calendar month only
■
▼
[Voucher Integrity] ■■■■■■> Never split lines sharing the same VOUCHER number across batches
■
▼
[Batch Creation] ■■■■■■■■■> Group into max 1000 lines per batch -> Save in MongoDB as PENDING
```

Detailed Step-by-Step Execution:

1. **Request Ingestion:** The client sends an Excel file payload to **`/api/v1/data-batch/upload`** along with the selected **`entryProcessorType`** (e.g. **`AccountReceivableFreight`**).
2. **Processor Lookup:** **`EntryProcessorFactory`** resolves the matching processor instance (e.g. **`AccountReceivableFreightEntryProcessor`**).
3. **Data Parsing & Normalization:** **`exceljs`** reads raw rows into strongly typed models (**`VendorEntryRawDataModel`** or **`AccountReceivableRawDataModel`**).
4. **Dimension Verification:** The pipe-separated dimension string (**`MainAccount|BusinessUnit|CostCenter|Activity...`**) is parsed. The main account and financial tags are checked against the pre-fetched **`MasterData`** cache.
5. **Business Rule Enforcement:**

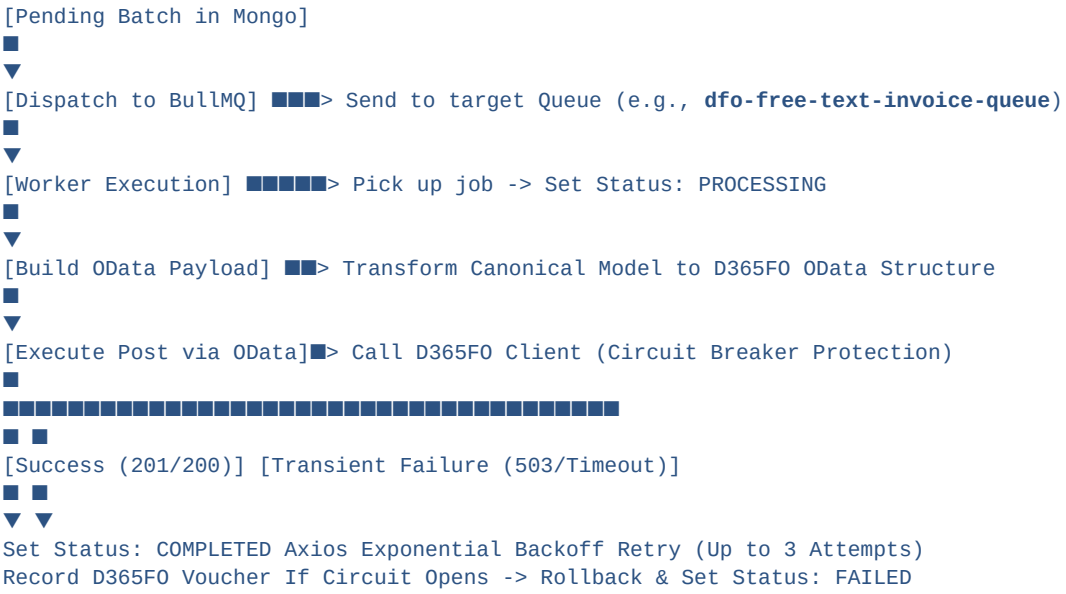
- **1000-Line Cap:** If a file contains 2,500 rows, it is split into 3 batches (e.g., 1000 + 1000 + 500).
- **Voucher Integrity:** If voucher **`VCH-1002`** spans lines 998–1003, the batch cap is dynamically adjusted so all lines of **`VCH-1002`** stay in the same batch.
- **Month Isolation:** Lines from January and February in the same sheet are split into separate batches to prevent month-end posting period errors in D365FO.

6. **MongoDB CQRS Storage:** Batches are saved with status **`DataBatchStatus.Pending`** in MongoDB.

■ Cycle 2: Asynchronous ERP Queue Posting & Resilience Cycle

Real-World Business Scenario

Batches stored in MongoDB must be posted to D365FO without locking the user interface or crashing if D365FO experiences temporary network latency.



Detailed Step-by-Step Execution:

1. **Queue Dispatch:** **`DataBatchService`** pushes pending batch IDs into the relevant BullMQ queue (e.g., **`dfo-vendor-journal-queue`**).
2. **Worker Processing:** BullMQ worker picks up the job, updates MongoDB batch status to **`Processing`**, and instantiates the posting strategy (**`VendorJournalPostingStrategy`**).
3. **Payload Mapping:**

- For Free Text Invoices: Generates **`FreeTextInvoiceHeaders`** and child **`FreeTextInvoiceLines`**.
- For Vendor Bills: Generates **`LedgerJournalHeaders`** (Journal Header) and **`LedgerJournalLines`** (Vendor & Ledger lines).

4. **Resilient HTTP Posting:**

- Calls **`D365FOClientService`** which wraps Axios calls in an **Opossum Circuit Breaker**.
- If D365FO returns **`503 Service Unavailable`** or times out, **`axios-retry`** waits exponentially (e.g., 2s, 4s, 8s) before retrying.

5. **Success Finalization:** On success, D365FO returns generated journal batch numbers / vouchers. MongoDB batch status is updated to **`Completed`**, recording D365FO identifiers.

6. **Failure & Rollback:** If posting fails permanently, the batch status is set to **`Failed`**, saving full error stack trace for admin exception handling.

■ Cycle 3: Master Data Synchronization & Caching Cycle

Real-World Business Scenario

To ensure rapid validation and eliminate N+1 OData round-trips during peak posting, D365FO master data (Chart of Accounts, Vendors, Customers, Tax Groups, Exchange Rates) is cached locally.

```
[Scheduled Cron / Manual Trigger]
■
▼
[MasterDataSyncQueue]
■
▼
[Query D365FO OData Entities] (MainAccounts, Vendors, Customers, ExchangeRates)
■
▼
[Transform & Normalize]
■
▼
[Store in Multi-Layer Cache] ■■> L1: Memory Cache
L2: Redis (Fast Lookup)
L3: MongoDB (Persistent Fallback)
```

Detailed Step-by-Step Execution:

1. **Trigger:** Executed automatically via NestJS **`SchedulerModule`** cron job or manually via **`/api/v1/master-data/sync`**.
2. **OData Extraction:** Uses **`ODataQueryBuilderService`** to pull active **`MainAccounts`**, **`Vendors`**, **`Customers`**, **`TaxGroups`**, and **`ExchangeRates`**.
3. **Multi-Tier Cache Storage:**

• **L1 (In-Memory):** Microsecond access for immediate validation.

• **L2 (Redis):** Shared across multiple running middleware Docker containers.

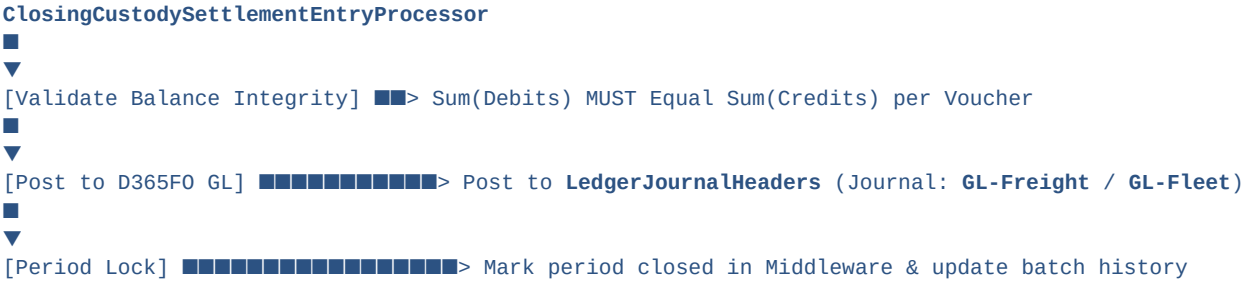
• **L3 (MongoDB):** Ensures cache availability even after system restarts.

■ Cycle 4: Month-End Period Closing & Reconciliation Cycle

Real-World Business Scenario

At the end of each financial period, finance teams perform period-end adjustments, balance clearing accounts (**`GL-Freight`**, **`GL-Fleet`**), reconcile driver custody advances, and close the period in D365FO.

```
[Month-End Cut-off]
■
▼
[Upload Period Adjustment Excel]
■
▼
[Closing Entry Processors] ■■■> ClosingFreightEntryProcessor
ClosingTruckingEntryProcessor
```



Detailed Step-by-Step Execution:

1. **Closing Sheet Upload:** Finance uploads GL period closing sheets containing main account balancing entries.
2. **Voucher Balance Check:** The processor calculates total debits and credits per voucher. If a voucher is imbalanced (`Debit != Credit`), the batch is rejected immediately with a validation error.
3. **Custody Clearing:** Internal custody advances paid to drivers are matched against actual trip expense vouchers. Net differences are moved to expense or settlement main accounts.
4. **Posting to GL:** The batch is posted to D365FO General Ledger journal `GL-Freight` or `GL-Fleet`.